

Apache Spark 101: Build, Transform, Count

A Practical
Introduction to
Big Data With
Python

Εργ. Βοηθός: Αντώνης Τζιόρβας
Email: atzio@unipi.gr

Why Distributed Analytics Matters Now



1 B Records Processed
Daily by Fortune-500
Firms



10× Faster ETL With
Spark Vs. Pandas on a
Single Node



Spark Powers the Data
Lake Of Netflix, Uber,
Airbnb

Install & Configure Spark (1)

The easy way

❖ To install Spark we need 4 things:

- ❖ Java
- ❖ Spark
- ❖ Python (+ Libraries)
- ❖ Hadoop (optional)



Install & Configure Spark (2)

The easy way

Installing & Configuring Java—Fast & “Fun”

1. Get [SDKMAN](https://sdkman.io) – the **Swiss-Army** knife for JDK on Unix
»» `curl -s "https://get.sdkman.io" | bash`
2. Pick your favorite JDK
»» `sdk install java 11.0.28-amzn`
3. Switch between versions in seconds
»» `sdk use java 11.0.28-amzn`
4. Verify the installation
»» `java -version`



Install & Configure Spark (3)

The easy way

Installing Spark (+ Hadoop)

Similarly to installing java

```
>>> sdk install spark
```

```
>>> sdk install hadoop [optional]
```

```

sdk install spark  ✓ 18s 13:51:18
Downloading: spark 3.5.3
In progress...
##### 100.0%
Repackaging Spark 3.5.3...
Done repackaging...
Installing: spark 3.5.3
Done installing!

Setting spark 3.5.3 as default.
✓ 34s 13:51:56
```

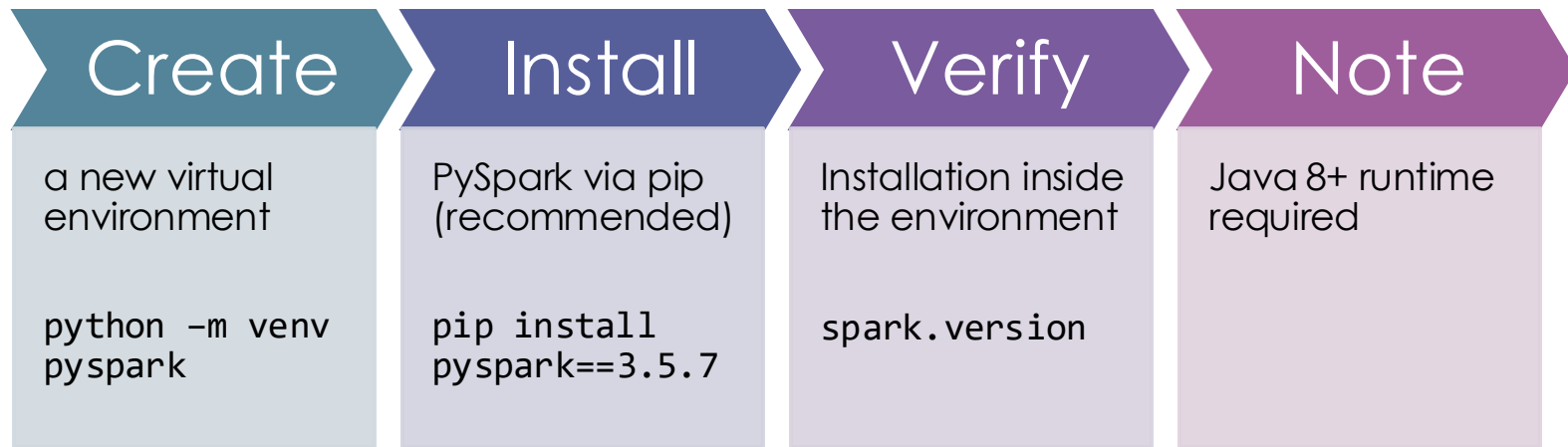


**Follow the
Documentation**



Follow Me

Kick-Off: Set Up a Local PySpark Environment in 5 min



SparkSession: The Entry Point for all Operations

- ❖ SparkContext → RDD API (low-level)
- ❖ SparkSession → unified entry point for SQL & DataFrames
- ❖ Configurable options (master, memory, cores)
- ❖ Implicitly creates a SparkContext for RDD APIs
- ❖ Show Spark UI URL (e.g., <http://localhost:4040>)
- ❖ Clean shutdown with `spark.stop()`

```
from pyspark import SparkContext  
from pyspark.sql import SparkSession
```

```
spark = (  
    SparkSession.builder.appName("SparkDemo")  
        .getOrCreate()  
)
```

```
sc = spark.sparkContext  
print("Spark version:", spark.version)  
print("App name: ", sc.appName)  
print("Master: ", sc.master)
```

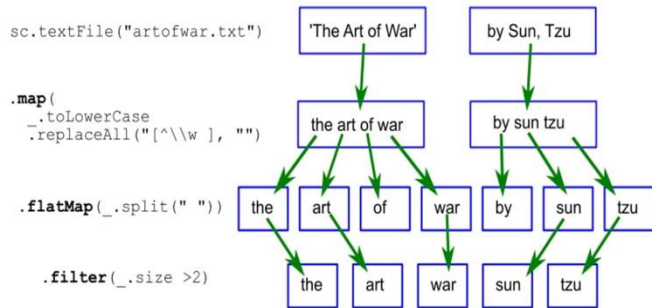
Resilient Distributed Datasets (RDDs) – Core Spark Data Structure

- ❖ Immutable – transformations produce new RDDs.
- ❖ Divided into logical partitions; each partition runs on a separate executor node.
- ❖ Created lazily from stable storage or from other RDDs via `map` / `filter` / `flatMap`
- ❖ Enables parallel “Map-Reduce-like” computations with in-memory speed-ups.
- ❖ Automatically recovers from node failures by recomputing lost partitions using lineage.
- ❖ Can be persisted (cached) in memory or on disk to reuse across actions.
- ❖ Transformations are lazy; actions (`count`, `collect`, etc.) trigger execution.



Spark: Visualizing RDD transformations

[http://sciabarra.com
/blog/spark-visualizing-rdd](http://sciabarra.com/blog/spark-visualizing-rdd)



Accumulators: Safe Counters for Distributed Metrics

- ❖ Define accumulator: `acc = sc.accumulator(0)`
- ❖ Increment inside transformations (e.g., malformed line counter)
- ❖ Readable only on driver after an action
- ❖ Common use-case: logging errors, progress tracking
- ❖ Pitfall: updates inside map are not guaranteed without an action

```
bad_cnt = sc.accumulator(0)
```

```
def parse(line):  
    if not valid(line):  
        bad_cnt.add(1)  
    return line
```

```
logs = sc.textFile('logs.txt').map(parse)  
logs.count()  
print('Bad lines:', bad_cnt.value)
```

Broadcast Variables: Efficient Read-Only Look-Ups

- ❖ Load small lookup table (e.g., sentiment dictionary)
- ❖ Broadcast with `sc.broadcast(dict_obj)`
- ❖ Access inside tasks via `.value`
- ❖ Reduces network shuffling vs. joining large tables
- ❖ Example: map each word to its sentiment score

```
stop_words =  
sc.broadcast(load_stopwords("./data/corpora/stopwords/english"))  
  
words = (  
    sc.textFile("data.txt")  
        .flatMap(lambda l: l.split())  
        .filter(lambda w: w not in stop_words.value)  
    )  
  
print(words.take(5))
```

Core RDD Transformations (1) – *map, flatMap, filter, sample*

- ❖ *map(f)* – applies *f* to every element
- ❖ *flatMap(f)* – *f* returns an iterable; all results are flattened
- ❖ *filter(p)* – keeps elements that satisfy predicate *p*
- ❖ *sample(withReplacement, fraction)* – randomly selects a fraction of the RDD

```
line_lengths = lines.map(lambda l: len(l))
```

```
tokens = lines.flatMap(lambda l: l.split())
```

```
long_words = tokens.filter(lambda w: len(w) > 5)
```

```
sample = long_words.sample(False, 0.01, seed=42)
```

Core RDD Transformations (2) – *reduce, fold, aggregate*

- ❖ *reduce(f)* – aggregates all elements using associative *f*
`reduce(lambda x, y: x + y)`
- ❖ *fold(zero, f)* – like reduce but uses zero for empty partitions
`fold(0, lambda x, y: x + y)`
- ❖ *aggregate(zero, seqOp, combOp)* – two-stage aggregation for complex stats

```
rdd = sc.parallelize([np.random.randint(0, 50) for _ in range(10)])
```

```
# reduce - sum of all elements
```

```
total = rdd.reduce(lambda a, b: a + b)
```

```
# fold - same as reduce but with an explicit zero value
```

```
folded = rdd.fold(0, lambda a, b: a + b)
```

```
result = data.aggregate(zeroValue, seqOp, combOp)
```

Set-Based Transformations – *union, intersection, subtract*

- ❖ *A.union(B)* – concatenates two RDDs
- ❖ *A.intersection(B)* – keeps only common elements
- ❖ *A.subtract(B)* – removes elements present in other

```
rddA = sc.parallelize([1,2,3])
```

```
rddB = sc.parallelize([3,4])
```

```
print(rddA.union(rddB).collect())
```

```
print(rddA.intersection(rddB).collect())
```

```
print(rddA.subtract(rddB).collect())
```

Key-Based & Join Transformations – reduceByKey, groupByKey, joins

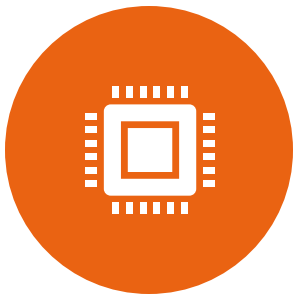
- ❖ *reduceByKey(f)* – merges values with same key using associative *f*. It aggregates locally first → 30 % less network I/O
- ❖ *groupByKey()* – groups all values per key (less efficient – shuffles all values)
- ❖ *join(other)* – inner join on keys
- ❖ *leftOuterJoin(other)* – left outer join, keeping all keys of the first RDD
- ❖ *cogroup(other)* – returns (key, (list1, list2)) per key

```
pairs = sc.parallelize([('a',1), ('b',2), ('a',3)])

print(pairs.reduceByKey(lambda x,y: x+y).collect())
# [('a',4), ('b',2)]

print(pairs.groupByKey().mapValues(list).collect())
# [('a',[1,3]), ('b',[2])]
```

Persisting RDDs – cache vs persist



`Cache()` – stores RDD in memory only (default)



`persist(storagelevel)` –
choose storage level
(`MEMORY_ONLY`, `DISK_ONLY`,
`MEMORY_AND_DISK`, ...)



Use before expensive repeated actions
(e.g.,
`rdd.persist(storagelevel.MEMORY_AND_DISK)` before `count()` & `collect()`)

RDD Actions – count, collect, take, first, foreach

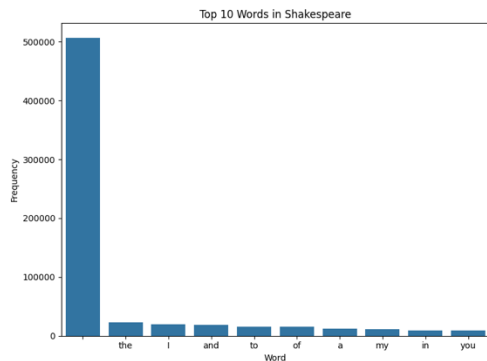
- ❖ *count()* – returns number of elements
- ❖ *collect()* – brings all elements to the driver (be careful with large RDDs)
- ❖ *take(n)* / *first()* – fetch first n elements (driver-side)
- ❖ *foreach(f)* – executes f on each element on executors (no return to driver)
- ❖ Use *foreachPartition* to write partitions to HDFS or external stores

```
print(rdd.count())  
print(rdd.take(5))  
print(rdd.first())
```

```
# Write each partition to a file  
rdd.foreachPartition(lambda it:  
    write_to_hdfs(it))
```

Word-Count End-to-End – From Text to Insights

- ❖ Create SparkSession
- ❖ Load Shakespeare (≈ 5 MB)
- ❖ Compute final top-10 words with frequencies
- ❖ Visualize result using matplotlib



```
from pyspark.sql import SparkSession
import matplotlib.pyplot as plt
```

```
spark = SparkSession.builder.getOrCreate()
lines = spark.read.text('shakespeare.txt').rdd
words = lines.flatMap(lambda row: row.value.split())
counts = words.map(lambda w: (w,
1)).reduceByKey(lambda a, b: a + b)
```

```
top10 = counts.takeOrdered(10, key=lambda x: -x[1])
labels, values = zip(*top10)
```

```
plt.bar(labels, values)
plt.title('Top 10 Words in Shakespeare')
plt.show()
```

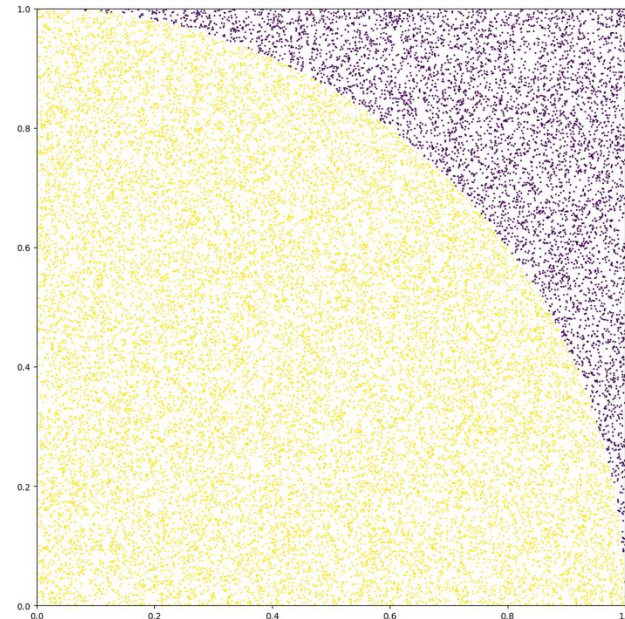
Monte-Carlo π Approximation – Generate Random Points

❖ Monte Carlo methods tend to follow a particular pattern:

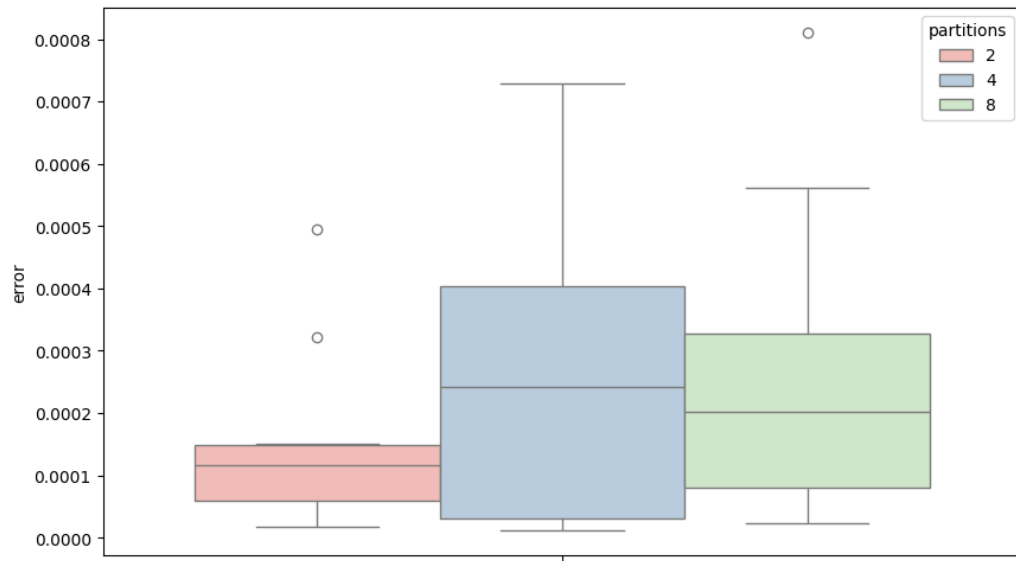
1. Define a domain of possible inputs.
2. Generate inputs randomly from a [probability distribution](#) over the domain.
3. Perform a [deterministic](#) computation of the outputs.
4. Aggregate the results.

❖ Important considerations:

1. If the points are not uniformly distributed, the approximation will be poor.
2. The approximation improves as more points are randomly placed in the whole square.



Tune Partitions – How Parallelism Affects π Accuracy



Wrap-Up Part A – What We Built & What's Next

Spark's core components (RDD, SQL, MLlib, GraphX, Streaming) and their roles

Key concepts: Accumulators, Broadcast Variables, and the transformation–action pipeline

Hands-on mini-projects: Word-Count & Monte-Carlo π demonstrating RDD API and performance tuning.

Next lesson:

Spark SQL, DataFrames, and Structured Streaming – diving deeper into declarative analytics.